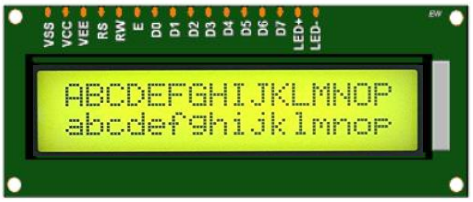
PROJETO2 AV2 – USO DO LCD SHIELD 16x2 COM TECLADO E SERIAL

1. Introdução ao LCD

Os display de cristal líquido (LCD) de texto são baseados no controlador HITACHI HD44780.
O controlador e o driver do LCD de matriz de pontos HD44780U exibe caracteres alfanuméricos, caracteres japoneses kana e símbolos. Ele pode ser configurado para controlar um display de cristal líquido de matriz de pontos sob o controle de um microprocessador de 4 ou 8 bits. Como todas as funções, como a RAM de exibição, o gerador de caracteres e o driver de cristal líquido necessários para controlar um display de cristal líquido de matriz de pontos, são fornecidas internamente em um único chip, um sistema mínimo pode ser conectado a este controlador/driver.

Um único HD44780 pode exibir até uma linha de 8 caracteres ou duas linhas de 8 caracteres. A ROM do gerador de caracteres do HD44780U é expandido para gerar 208 fontes de caracteres de 5 × 8 pontos e 32 fontes de caracteres de 5 × 10 pontos, totalizando 240 fontes de caracteres diferentes.

A função de cada pino no LCD pode ser vista na Figura1:



No.	PIN	Function
1	VSS	Ground
2	VCC	+5 Volt
3	VEE	Contrast control 0 Volt: High contrast.

No.	PIN	Function
4	RS	Register Select 0: Commannd Reg. 1: Data Reg.
5	RW	Read / write 0: Write 1: Read
6	E	Enable H-L pulse
7-14	D0 - D7	Data Pins D7: Busy Flag Pin
15	LED+	+5 Volt
16	LED-	Ground

EW

Figura 1 – Pinagem do LCD 16x2

Pino 1:2 - Alimentação

Pino 3 - Pino VEE

Este pino é usado para ajustar o contraste do display. A tensão neste pino define o contraste no display, quanto menor a tensão, maior o contraste. Podemos conectar um potenciômetro de 4,7k para ajuste de contraste ou simplesmente conectar este pino ao terra para obter o contraste máximo.

Pino 4 - RS: Pino de Seleção de Registro

RS = 0: Dados nos pinos D0 a D7 são considerados como um comando.
RS = 1: Dados nos pinos D0 a D7 são considerados como dados para exibir no LCD16x2.

Pino 5 - RW: Pino de Leitura/Escrita

RW = 0: Escrever dados no LCD.
RW = 1: Ler dados do LCD.

Pino 6 - E: Habilitar

Este pino é usado para travar os dados presentes nos pinos de dados D0 a D7. Um pulso alto-baixo com uma largura mínima de 450 ns é necessário para *latchar* os dados no display.

Pinos 7:14 - Pinos de Dados D0 a D7

Os pinos de dados são usados para enviar dados/comandos ao LCD16x2 como 8 bits de dados paralelos.

Pin 15:16 - LED + e LED -

Os Displays de Cristal Líquido não possuem sua própria luz como os displays de sete segmentos. Portanto, o módulo tem um LED para obtenção da luz de fundo. O suprimento para este LED é fornecido através desses pinos.

1.1 Modos de operação e funções do LCD

Um circuito interno de inicialização inicializa automaticamente o HD44780U quando a energia é ligada. As seguintes instruções são executadas durante a inicialização. O indicador de ocupado Busy Flag (BF) é mantido no estado ocupado até que a inicialização termine (BF = 1). O estado ocupado dura 10 ms após o VCC subir para 4,5 V.

Verificando o BF

Para verificar o estado do sinal de ocupado e ler o contador de endereço:

- Configure o pino R/W do LCD como HIGH (leitura do LCD)
- Selecione o registrador de instrução definindo o pino RS como LOW
- Ative o LCD definindo o pino *enable* como HIGH
- O bit mais significativo do barramento de dados do LCD é o estado do sinal de ocupado (1=Ocupado, 0=pronto para aceitar instruções/dados). Os outros bits mantêm o valor atual do contador de endereço.

REFERÊNCIA DE COMANDOS

Instruction	Code RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Description	Execution Time (max) (when f_{cp} or f_{OSC} is 270 kHz)
Clear display	0	0	0	0	0	0	0	0	0	1	Clears entire display and sets DDRAM address 0 in address counter.	
Return home	0	0	0	0	0	0	0	0	1	—	Sets DDRAM address 0 in address counter. Also returns display from being shifted to original position. DDRAM contents remain unchanged.	1.52 ms
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction and specifies display shift. These operations are performed during data write and read.	37 µs
Display on/off control	0	0	0	0	0	0	1	D	C	B	Sets entire display (D) on/off, cursor on/off (C), and blinking of cursor position character (B).	37 µs
Cursor or display shift	0	0	0	0	0	1	S/C	R/L	—	—	Moves cursor and shifts display without changing DDRAM contents.	37 µs
Function set	0	0	0	0	1	DL	N	F	—	—	Sets interface data length (DL), number of display lines (N), and character font (F).	37 µs
Set CGRAM address	0	0	0	1	ACG	ACG	ACG	ACG	ACG	ACG	Sets CGRAM address. CGRAM data is sent and received after this setting.	37 µs
Set DDRAM address	0	0	1	ADD	ADD	ADD	ADD	ADD	ADD	ADD	Sets DDRAM address. DDRAM data is sent and received after this setting.	37 µs
Read busy flag & address	0	1	BF	AC	AC	AC	AC	AC	AC	AC	Reads busy flag (BF) indicating internal operation is being performed and reads address counter contents.	0 µs
Write data to CG or DDRAM	1	0	Write data								Writes data into DDRAM or CGRAM.	37 µs t_{ADD} = 4 µs*
Read data from CG or DDRAM	1	1	Read data								Reads data from DDRAM or CGRAM.	37 µs t_{ADD} = 4 µs*
	I/D = 1: Increment I/D = 0: Decrement S = 1: Accompanies display shift S/C = 1: Display shift S/C = 0: Cursor move R/L = 1: Shift to the right R/L = 0: Shift to the left DL = 1: 8 bits, DL = 0: 4 bits N = 1: 2 lines, N = 0: 1 line F = 1: 5 × 10 dots, F = 0: 5 × 8 dots BF = 1: Internally operating BF = 0: Instructions acceptable										DDRAM: Display data RAM CGRAM: Character generator RAM ACG: CGRAM address ADD: DDRAM address (corresponds to cursor address) AC: Address counter used for both DD and CGRAM addresses	Execution time changes when frequency changes Example: When f_{cp} or f_{OSC} is 250 kHz, $37\ \mu s \times \frac{270}{250} = 40\ \mu s$

Tabela 1 – Lista de comandos do LCD16x2

Descrição das Instruções

- Limpar o Display

Limpar o display escreve o código de espaço 20H (padrão de caractere para o código de caractere 20H deve ser um padrão em branco) em todos os endereços DDRAM. Ele então define o endereço DDRAM 0 no contador de endereço e retorna o display para seu status original se foi deslocado. Em outras palavras, o display desaparece e o cursor ou o piscar vai para a borda esquerda do display (na primeira linha se duas linhas forem exibidas). Ele também define I/D como 1 (modo de incremento) no modo de entrada. S do modo de entrada não muda.

- Retornar ao Início - Home

Retornar ao início define o endereço DDRAM 0 no contador de endereço e retorna o display ao seu status original se foi deslocado. O conteúdo do DDRAM não muda. O cursor ou o piscar vai para a borda esquerda do display (na primeira linha se duas linhas forem exibidas).

- Configurar Modo de Entrada

I/D: Incrementa (I/D = 1) ou decrementa (I/D = 0) o endereço DDRAM em 1 quando um código de caractere é escrito ou lido do DDRAM.
O cursor ou o piscar se move para a direita quando incrementado em 1 e para a esquerda quando decrementado em 1. O mesmo se aplica à escrita e leitura do CGRAM.
S: Desloca todo o display para a direita (I/D = 0) ou para a esquerda (I/D = 1) quando S é 1. O display não se move se S for 0.
Se S for 1, parecerá como se o cursor não se move, mas o display se move. O display não se desloca ao ler do DDRAM. Além disso, a escrita ou leitura do CGRAM não desloca o display.

- Controle de Ligado/Desligado do Display

D: O display está ligado quando D é 1 e desligado quando D é 0. Quando desligado, os dados de display permanecem no DDRAM, mas podem ser exibidos instantaneamente definindo D como 1.
C: O cursor é exibido quando C é 1 e não é exibido quando C é 0. Mesmo que o cursor desapareça, a função de I/D ou outras especificações não mudará durante a gravação de dados de exibição. O cursor é exibido usando 5 pontos na 8ª linha para seleção de fonte de caractere de 5 × 8 pontos e na 11ª linha para a seleção de fonte de caractere de 5 × 10 pontos.
B: O caractere indicado pelo cursor pisca quando B é 1. O piscar é exibido como a alternância entre todos os pontos em branco e caracteres exibidos em uma velocidade de intervalos de 409,6 ms quando fcp ou fOSC é 250 kHz. O cursor e o piscar podem ser definidos para exibir simultaneamente. (A frequência de piscar muda de acordo com fOSC. Por exemplo, quando fcp é 270 kHz, 409,6 × 250/270 = 379,2 ms.)

- Cursor or Display Shift (Deslocamento do Cursor ou da Tela)

Desloca a posição do cursor ou do display para a direita ou esquerda sem escrever ou ler dados no display (Tabela 1). Essa função é usada para corrigir ou buscar informações no display. Em um display de 2 linhas, o cursor se move para a segunda linha quando passa do 40º caractere da primeira linha. Note que no deslocamento do display a primeira e segunda linha se deslocam ao mesmo tempo. Quando os dados exibidos são deslocados repetidamente, cada linha se move apenas horizontalmente. A exibição da segunda linha não se desloca para a posição da primeira linha. O conteúdo do contador de endereço (AC) não mudará se a única ação realizada for um deslocamento do display.

- Função Set

DL: Define o comprimento dos dados de interface. Os dados são enviados ou recebidos em comprimentos de 8 bits (DB7 a DB0) quando DL é 1 e em comprimentos de 4 bits (DB7 a DB4) quando DL é 0. Quando o comprimento de 4 bits é selecionado, os dados devem ser enviados ou recebidos duas vezes.

N: Define o número de linhas da tela.

F: Define o tipo de fonte de caracteres.

Observação: Execute a função no início do programa antes de executar qualquer instrução (exceto para ler o BF e a instrução de endereço). A partir desse ponto, a instrução de

configuração da função não pode ser executada a menos que o comprimento dos dados da interface seja alterado.

2. Implementação Prática

Nessa prática, iremos usar a placa Shield para Arduino integrada a placa NÚCLEO. A placa STM32NÚCLEO possui um barramento de pinos compatível com Arduino onde o Shield LCD pode ser conectado. O Shield LCD para Arduino é mostrado na Figura 2:

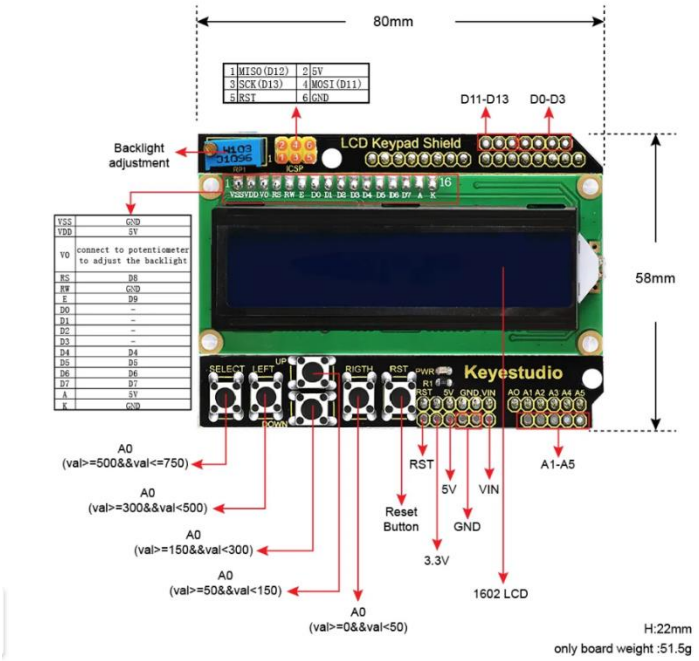


Figura 2 - Shield LCD para Arduino

O Shield possui um LCD 16x2 e algumas teclas que possibilitam aplicações com implementações de uma interface homem-máquina (IHM). Essas teclas podem ser utilizadas como entrada em um menu de opções em determinadas aplicações como, por exemplo, as opções sugeridas na própria serigrafia da placa referente as teclas [SELECT, LEFT, UP, DOWN, RIGHT]. Além dessas, existe uma tecla extra referente ao RESET do sistema. No caso da placa NÚCLEO, a tecla RESET irá resetar o microcontrolador usado na placa.

O circuito das teclas e de interface com o display é mostrado na Figura3. Veja que o circuito do teclado utiliza uma rede resistiva para forma um divisor de tensão, de forma que, a tensão presente no ponto A0 será função da tecla pressionada. Essa é uma técnica simples para identificar uma tecla pressionada, pois é necessário somente ler o valor da tensão no ponto A0. Isso pode ser feito de maneira fácil usando um dos canais de entrada do conversor AD do microcontrolador da placa NÚCLEO para ler o potencial presente em A0. Observe que, nesse caso, se consumiu apenas um GPIO do micro para a leitura de todas as teclas. Pode ser observado também no manual da placa Núcleo que, de acordo com o circuito esquemático de interface do LCD mostrado na

Figura3, o ponto A0 da placa Shield está conectado a entrada analógica A1 na placa Núcleo. Portanto, será necessário usar o canal 1 do ADC para leitura e decodificação da tecla pressionada. Outro aspecto importante a ser observado na interface com o LCD é a ligação do barramento de dados do LCD, que nesse caso, utiliza uma interface de 4 bits e não de 8 bits.

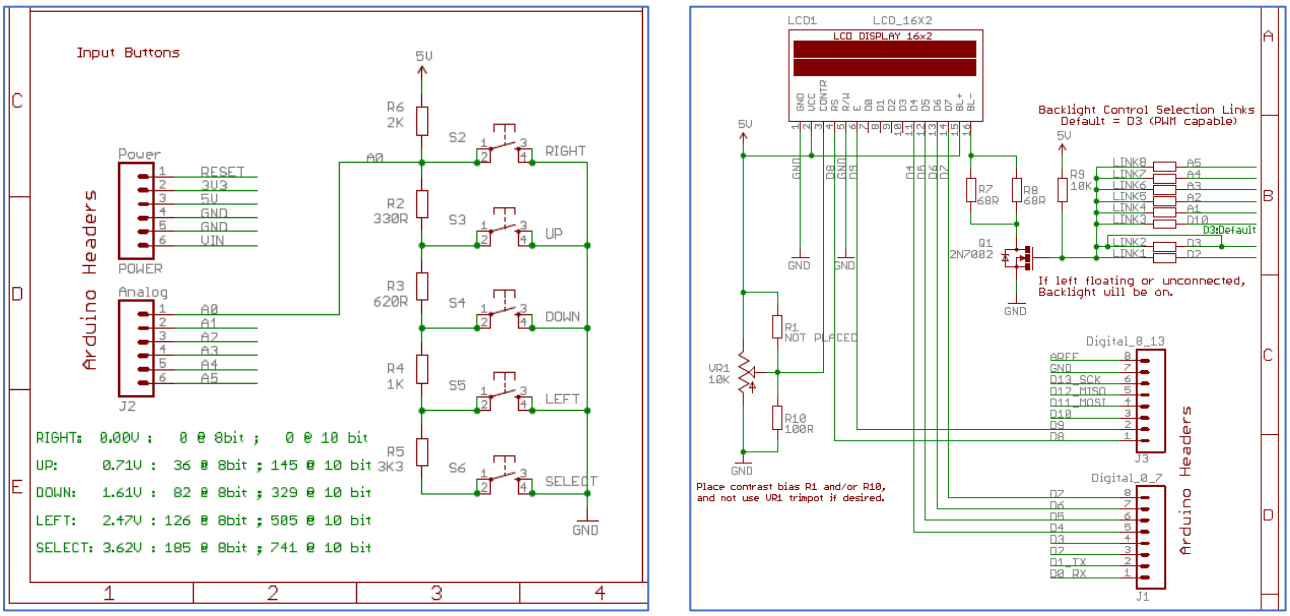


Figura3 – Interface do teclado e interface do LCD da placa Shield LCD para Arduino

PARTE 1 – Configuração e envio de dados e comandos para o LCD

1. Criar a função de inicialização dos pinos do LCD [E, RS, D7,D6,D5,D4]. Nesse caso, os sinais correspondente no STM32Fxx são [E->GPIOC7, RS->GPIOA9, D7->GPIOA8, D6->GPIOB10, D5->GPIOB4, D4->GPIOB5]. Todos os pinos devem ser configurados como saídas e os respectivos barramentos das portas devem ser habilitados;
2. Criar a função de escrever no LCD (dado ou comando);
3. Criar a função de enviar um comando para o LCD. Nesse caso, atentar para o nível lógico necessário para os sinais RS = 0 e E=1;
4. Criar a função de inicialização do LCD. Essa função deve prever a inicialização do LCD no modo de operação com 4 bits. Atentar para as temporizações envolvidas nesse processo. Devem ser feitas pesquisas no datasheet ou na internet sobre o processo de inicialização do LCD;
5. Criar a função de enviar um dado (caractere) para o LCD. Nesse caso, atentar para o nível lógico necessário para os sinais RS = 1 e E=1;
6. Criar uma função para limpar o LCD;
7. Criar uma função para posicionar o cursor em uma linha e coluna específica do LCD;
8. Criar uma função para enviar uma *string* para o LCD;

PARTE 2 – Leitura e decodificação das teclas

1. Usar os códigos já implementados anteriormente referente ao conversor AD do STM32F4xx para ler o valor de tensão analógica presente no canal A1 e fazer a decodificação da tecla correspondente. A tecla lida deve ser mostrada na linha 2 do LCD como texto da seguinte forma: [SELECT, LEFT, UP, DOWN, RIGHT].

PARTE 3 – Rotinas de temporização de delays grandes e pequenos

1. Usar um dos timer para a geração de delays. Sugestão é programar uma função delay baseada em 1 mS. Exemplo de função:
   delay (3000); -> corresponde a uma chamada de um delay de 3s;
2. Usar uma estrutura de delay baseado em FOR para a geração de delays pequenos para inicialização e uso no LCD;

PARTE 4 – Aplicação principal – Main

1. Após o reset, enviar a mensagem para o LCD conforme abaixo:
   LINHA 1 – SIST. TEMPO REAL
   LINHA 2 – * U N I F O R *
2. Gerar um delay de 3s;
3. Após 3s entrar na tela de espera de uma tecla:
   LINHA 1 – TECLA ACIONADA:
   LINHA 2 – NONE
   Enquanto pressionado uma tecla o sistema deve mostrar na linha 2 a tecla pressionada. Exemplo:
   LINHA 1 – TECLA ACIONADA:
   LINHA 2 – RIGHT
   Ao soltar a tecla volta para a tela anterior.

CONTINUA



Modelo de Solução

Atentar para a inclusão dos arquivos header .h necessários a cada parte do código.

PARTE 1 – Configuração e envio de dados e comandos para o LCD

1. Criar a função de inicialização dos pinos do LCD [E, RS, D7,D6,D5,D4]. Nesse caso, os sinais correspondente no STM32Fxx são [E->GPIOC7, RS->GPIOA9, D7->GPIOA8, D6->GPIOB10, D5->GPIOB4, D4->GPIOB5]. Todos os pinos devem ser configurados como saídas e os respectivos barramentos das portas devem ser habilitados. Atente para as temporizações envolvidas nesse processo. Devem ser feitas pesquisas no datasheet ou na internet sobre o processo de inicialização do LCD;

```
28 /************************************************************
29 void GPIO_init(void)
30 {
31     //Habilita clock para GPIOA, GPIOB, GPIOC
32     RCC->AHB1ENR |= (1U<<0) | (1U<<1) | (1U<<2);
33
34     GPIOA->MODER |= (1U<<18);  // Pino 9 como saída - RS
35     GPIOA->MODER &=~(1U<<19);
36
37     GPIOC->MODER |=(1U<<14);   // Pino 7 como saída - EN
38     GPIOC->MODER &=~(1U<<15);
39
40     GPIOB->MODER |=(1U<<10);   // Pino 5 como saída - D4
41     GPIOB->MODER &=~(1U<<11);
42
43     GPIOB->MODER |=(1U<<8);    // Pino 4 como saída - D5
44     GPIOB->MODER &=~(1U<<9);
45
46     GPIOB->MODER |=(1U<<20);   // Pino 10 como saída - D6
47     GPIOB->MODER &=~(1U<<21);
48
49     GPIOA->MODER |=(1U<<16);   // Pino 8 como saída - D7
50     GPIOA->MODER &=~(1U<<17);
51
52     GPIOA->MODER |=(1U<<10);   //LED
53     GPIOA->MODER &=~(1U<<11);
54 }
```

2. Criar a função de escrever no LCD (dado ou comando);

```
56 /************************************************
57 void send_to_lcd (int data, int rs)
58 {
59 //   rs = 1 para dados, rs=0 para comando
60     if (rs == 1) {
61         GPIOA->ODR |=  (1U<<9);}
62         else if (rs == 0){
63         GPIOA->ODR &= ~(1U<<9);
64         }
65
66     if (((data>>3)&0x01) == 1) {
67         GPIOA->ODR |=  (1U<<8);}    //d7 pin
68         else {
69         GPIOA->ODR &= ~(1U<<8);
70         }
71
72     if (((data>>2)&0x01) == 1) {
73         GPIOB->ODR |=  (1U<<10);}   //d6 pin
74         else {
75         GPIOB->ODR &= ~(1U<<10);
76         }
77
78     if (((data>>1)&0x01) == 1) {
79         GPIOB->ODR |=  (1U<<4);}    //d5 pin
80         else {
81         GPIOB->ODR &= ~(1U<<4);
82         }
83
84     if (((data>>0)&0x01) == 1) {
85         GPIOB->ODR |=  (1U<<5);}    //d4 pin
86         else {
87         GPIOB->ODR &= ~(1U<<5);
88         }
89
90     // Habilita o enable do LCD
91     GPIOC->ODR |= (1U<<7);
92     delayLCD (100);
93
94     // Desabilita o enable do LCD
95     GPIOC->ODR &= ~(1U<<7);
96     delayLCD (100);
97 }
```

3. Criar a função de enviar um comando para o LCD. Nesse caso, atentar para o nível lógico necessário para os sinais RS = 0 e E=1;

```
99  void lcd_send_cmd (char cmd)
100 {
101     char datatosend;
102     /* Envia o nibble superior do byte */
103     datatosend = ((cmd>>4)&0x0f);
104     send_to_lcd(datatosend, 0);  // RS deve ser 0 enquanto enviando um comando
105
106     /* Envia o nibble inferior do byte */
107     datatosend = ((cmd)&0x0f);
108     send_to_lcd(datatosend, 0);
109 }
```

4. Criar a função de inicialização do LCD. Essa função deve prever a inicialização do LCD no modo de operação com 4 bits. Atentar para as temporizações envolvidas nesse processo. Devem ser feitas pesquisas no datasheet ou na internet sobre o processo de inicialização do LCD;

```
146 void lcd_init (void)
147 {
148     // Inicialização no modo 4 bits
149     delay_ms(50);  // Espera > 40ms
150     lcd_send_cmd (0x30);
151     delay_ms(5);  // Espera por > 4.1ms
152     lcd_send_cmd (0x30);
153     delay_ms(1);  // Espera por > 100us
154     lcd_send_cmd (0x30);
155     delay_ms(1);  // Espera por > 100us
156     lcd_send_cmd (0x20);  // mode 4 bits
157     delay_ms(1);  // Espera por > 100us
158
159   // inicialização do display
160     lcd_send_cmd (0x28); // Função set --> DL=0 (4 modo 4 bits), N = 1 (2 linhas) F = 0 ( Fonte 5x8)
161     delay_ms(50);
162     lcd_send_cmd (0x08); //Controle on/off do display --> D=0, C=0, B=0  ---> display off
163     delay_ms(50);
164     lcd_send_cmd (0x01);  // limpa display
165     delay_ms(50);
166     delay_ms(50);
167     lcd_send_cmd (0x06); //Entry mode set --> I/D = 1 (incrementa cursor) & S = 0 (não shift)
168     delay_ms(50);
169     lcd_send_cmd (0x0C); //Display on/off control --> D = 1, C e B = 0. (Cursor e blink, últimos 2 bits)
170 }
```

5. Criar a função de enviar um dado (caractere) para o LCD. Nesse caso, atentar para o nível lógico necessário para os sinais RS = 1 e E=1;

```
111 void lcd_send_data (char data)
112 {
113     char datatosend;
114
115     /* Envia o nibble superior do byte */
116     datatosend = ((data>>4)&0x0f);
117     send_to_lcd(datatosend, 1);  // RS deve ser 1 enquanto enviando um dado
118
119     /* Envia o nibble inferior do byte */
120     datatosend = ((data)&0x0f);
121     send_to_lcd(datatosend, 1);
122 }
```

6. Criar uma função para limpar o LCD;

```
124 void lcd_clear (void)
125 {
126     // comando 01 - Limpa LCD
127     lcd_send_cmd(0x01);
128     delay_ms(100);
129 }
```

7. Criar uma função para posicionar o cursor em uma linha e coluna específica do LCD;

```
131 void lcd_put_cur(int row, int col)
132 {
133     switch (row)
134     {
135         case 0:
136             col |= 0x80; // 0x80 = Endereço da 1a col. na linha 1
137             break;
138         case 1:
139             col |= 0xC0; // 0xC0 = Endereço da 2a col. na linha 1
140             break;
141     }
142
143     lcd_send_cmd (col);  // Posiciona o curso no endereço a ser
144                          // escrito no LCD
145 }
```

8. Criar uma função para enviar uma *string* para o LCD;

```
173 void lcd_send_string (char *str)
174 {
175     while (*str) lcd_send_data (*str++);
176 }
```

PARTE 2 – Leitura e decodificação das teclas

1. Usar os códigos já implementados anteriormente referente ao conversor AD do STM32F4xx para ler o valor de tensão analógica presente no canal A1 e fazer a decodificação da tecla correspondente. A tecla lida deve ser mostrada na linha 2 do LCD como texto da seguinte forma: [SELECT, LEFT, UP, DOWN, RIGHT].

```
 8 //      keypad.h
 9 #include "stdint.h"
10
11 #define KEY_RIGHT  0
12 #define KEY_UP     1
13 #define KEY_DOWN   2
14 #define KEY_LEFT   3
15 #define KEY_SELECT 4
16 #define KEY_NONE   5
17
18 uint8_t keypad_init();
19 uint16_t keypad_read_key();
20
```

```
 7 #include "stm32f4xx.h"
 8 #include "keypad.h"
 9
10 uint8_t keypad_init()
11 {
12     RCC->AHB1ENR |= (1U<<0);   // Habilita clock para GPIOA
13     RCC->APB2ENR |= (1U<<8);   // Habilita clock para ADC1
14
15     GPIOA->MODER |= 0x3;       // Configura PA0 como ent. analógica
16
17     ADC1->CR2  = 0;            // Dispara aquisição por software
18     ADC1->SQR3 = 0;            // Sequência de conversão começa
19                                // no canal 0
20     ADC1->SQR1 = 4;            // Tamanho da sequência de conversão 1
21     ADC1->CR2 |= 1;            // Habilita ADC1
22     return 0;
23 }
```

```c
uint16_t keypad_read_key()
{
    uint16_t adc_readout = 0;

    ADC1->CR2 |= (1U<<30);      // Inicia a conversão
    while (!(ADC1->SR & 2)){
    }                           // Espera o final da conversão
    adc_readout = ADC1->DR;     // Retorna o resultado

    if(adc_readout > 800 && adc_readout < 850)
    {
        return KEY_UP;
    }
    else if (adc_readout > 1950 && adc_readout < 2050)
    {
        return KEY_DOWN;
    }
    else if (adc_readout > 3050 && adc_readout < 3150)
    {
        return KEY_LEFT;
    }
    else if (adc_readout >= 0 && adc_readout < 50)
    {
        return KEY_RIGHT;
    }
    else if (adc_readout > 4000 && adc_readout < 5050)
    {
        return KEY_SELECT;
    }
    return KEY_NONE;
}
```

PARTE 3 – Rotinas de temporização de delays grandes e pequenos

3. Usar um dos timer para a geração de delays. Sugestão é programar uma função delay baseada em 1 mS. Exemplo de função:
   delay (3000); -> corresponde a uma chamada de um delay de 3s;
4. Usar uma estrutura de delay baseado em FOR para a geração de delays pequenos para inicialização e uso no LCD;

- Usar as definições e inicializações para o timer2 para gerar uma base de tempo de 1mS – Delay de 1mS. Não usar interrupção do timer

```c
#include "stm32f4xx.h"
#include "stdint.h"

#define TIM2EN      (1U<<0)
#define CR1_CEN     (1U<<0)
#define SR_UIF      (1U<<0)

void tim2_init(void)
{
    /* Habilita o clock para acesso ao timer2 */
    RCC->APB1ENR |=TIM2EN;
    /* Configura o valor do prescaler */
    TIM2->PSC =  16 ;  //  16.000.000 / 16 = 1.000.000
    /* Configura o valor de recarga automática (auto-reload) */
    TIM2->ARR =  1000 ;  // 1.000.000 / 1000 = 1000
    /* Limpa o contador */
    TIM2->CNT = 0;
    /* Desabilita o timer*/
    TIM2->CR1 &= ~CR1_CEN;
    /* Habilita a interrupção do TIM2*/
//  TIM2->DIER |=DIER_UIE;
    /* Habilita a interrupção do controlador para o TIM2*/
//   NVIC_EnableIRQ(TIM2_IRQn);
  }

```

Configuração do timer

```c
void delay (uint16_t ms)
{
    /* Habilita o timer*/
    TIM2->CR1 |= CR1_CEN;
    TIM2->CNT = 0;
    for (int i = 0; i < ms; i++) {
    /* Espera pelo flag UIF - Estouro do timer */
    while(!(TIM2->SR & SR_UIF)){}
        /* Limpa o flag UIF */
        TIM2->SR &=~SR_UIF;
     };
        TIM2->CR1 &= ~CR1_CEN;
}

// Delay de aprox. 1mS
void delay_ms (uint16_t us)
{
    for (int i = 0; i < 50*us; i++) {}
}

// Delay de aprox. 20uS
void delayLCD (uint16_t us)
{
    for (int i = 0; i < us; i++) {}
}
```

Rotinas de delays

PARTE 4 – Aplicação principal – Main

```
#include "stm32f4xx.h"
#include "LCD1602.h"
#include "keypad.h"
#include "stdio.h"
#include "tim.h"

int row=0;
int col=0;

int main(void)
{
  uint16_t key_val = 0;
  static char* key_name[] = {"RIGHT", "UP", "DOWN", "LEFT", "SELECT", "NONE  "};

  GPIO_init();
  tim2_init();

  keypad_init();

  lcd_init();
  lcd_put_cur(0,0);
  lcd_send_string("SIST. TEMPO REAL");

  lcd_put_cur(1, 0);
  lcd_send_string("* U N I F O R *");
  delay(3000);

  lcd_clear();
  lcd_put_cur(0, 0);
  lcd_send_string("TECLA ACIONADA:");

  int bounce;

    while (1)
      {
            key_val = keypad_read_key();

            if ((key_val == 4) & (bounce==0))
                {
                bounce++;
                }
            else{
                 lcd_put_cur(1, 0);
                 lcd_send_string(key_name[key_val]);
                 lcd_send_string("    ");
                 delay(100);
                 bounce = 0;
                 }
      }
}
```

ATIVIDADE AV2

1. Após o reset, enviar a mensagem para o LCD conforme abaixo:
   LINHA 1 – SIST. TEMPO REAL
   LINHA 2 – * U N I F O R *
2. Gerar um delay de 3s;
3. Após 3s entrar na tela de espera de uma tecla:
   LINHA 1 – TECLA ACIONADA:
   LINHA 2 – NONE
   Enquanto pressionado uma tecla o sistema deve mostrar na linha 2 a tecla pressionada. Exemplo:
   LINHA 1 – TECLA ACIONADA:
   LINHA 2 – RIGHT

Ao soltar a tecla volta para a tela anterior.

4. **Adicionar na coluna 7 da linha 2 do LCD um caractere X recebido pela serial:**
   LINHA 1 – TECLA ACIONADA:
   LINHA 2 – NONE X